HARRY OWIREDU-ASHLEY

My AWS Security Lab

How I built a small EC2 application, restricted its access to a private S3 object, tested the controls, and checked the audit evidence.

Public portfolio edition | October 2026

MY GOAL	WHAT I BUILT
Learn by doing	A Python API on EC2 that reads a text file from private S3 storage.
Keep access intentional	Separate human setup permissions, a narrowly scoped application role, and source-IP restrictions.
Check the evidence	Successful and denied requests, management-event records, and an S3 GetObject data event.
Finish the exercise	Saved evidence, removed the lab resources, and removed temporary setup assignments.

I wanted to understand what actually happens between opening a URL, reading a private file, and proving which AWS identity made that request. This report follows the lab from account setup through cleanup, including the errors I had to work through.

This was a guided learning project. I performed the configuration and tests with AI-assisted guidance, then checked the results in the console, terminal, and logs. I am documenting the work as a lab, not presenting it as a production deployment.

START HERE / 01

About this public version

I wrote this report to explain both what I did and why it mattered. I kept the language straightforward because understanding the relationship between the services was more useful to me than memorizing a sequence of console clicks.

How I handled evidence

• **Observed results:** I used terminal output, console screenshots, and downloaded CloudTrail records to check the behavior.

• **Cleanup notes:** the cleanup status reflects the actions I completed and recorded during the lab. It is not a new live inspection of the AWS account.

• **Public excerpts:** I included only selected log fields needed to explain the result. Identifiers are replaced with consistent example values; the excerpts are not untouched original logs.

I excluded billing details and the original screenshots from this public edition. The screenshots contain account or session context, so I retained the originals privately and described the relevant results in text. This avoids publishing hidden image details or relying on an opaque box placed over sensitive pixels.

The account numbers 111111111111 and 222222222222 are placeholders. Addresses 198.51.100.10 and 203.0.113.10 are documentation examples, not live endpoints. Resource identifiers, usernames, and bucket names are also sanitized examples. Names and IDs in this report must not be used as real deployment targets.

READING ROUTE	SECTIONS
Understand the design	2-7: architecture, accounts, identities, policies, and the application role
Build and test	8-18: storage, compute, networking, Python, and troubleshooting
Inspect the evidence	19-24: CloudTrail, selectors, the successful object read, and the timeline
Close the lab	25-27: cleanup, remaining access, and lessons learned
Look up terminology	28-31: glossary, command reference, and sources

The narrative uses Eastern Daylight Time (EDT) for October 15, 2026. AWS timestamps ending in Z are UTC, four hours ahead of EDT on that date. I retained the event times so the sequence remains understandable.

LAB WALKTHROUGH / 02

What I built: the complete picture

I built a small web application on an Amazon EC2 instance. A visitor could request a health check or a greeting. For the greeting, the application fetched a text file from a private S3 bucket. The application used an AWS role to read that file, rather than a password or permanent AWS access key stored in its code.

PART	PLAIN-LANGUAGE JOB
My Windows computer	The machine I used to sign in, connect with SSH, and test the API in a browser.
EC2 instance	A rented Linux computer in AWS that ran my Python application.
FastAPI and Uvicorn	FastAPI defined the routes; Uvicorn listened for web requests and ran the application.
LabAppRole	The application identity with permission to read one particular S3 object.
Private S3 data bucket	Stored hello.txt without making it publicly readable.
CloudTrail and audit bucket	Recorded selected AWS actions and stored evidence for later review.

What happened when I opened /hello

• 1. My browser sent an HTTP request to the EC2 public address on port 8080.

• 2. The security group checked whether the connection came from the allowed public IP address.

• 3. Uvicorn passed the request to the FastAPI /hello function.

• 4. Boto3 sent an authenticated GetObject request to S3 using temporary role credentials.

• 5. S3 checked the role permission and returned the contents of hello.txt.

• 6. The API put that text in a JSON response and sent it back to my browser.

There were two separate connections: browser to application, and application to S3. Keeping the S3 bucket private did not prevent the application from returning its contents to an allowed visitor. That is why the application also needed its own network restriction.

The API connection used HTTP in this lab. The recorded S3 request used TLS 1.3. Encryption on one connection does not automatically encrypt the other connection.

LAB WALKTHROUGH / 03

Accounts and root protection

I used a management account and a separate member account for the lab. An AWS account is a boundary for resources, permissions, and billing. AWS Organizations groups accounts together so they can be managed centrally. The management account was **111111111111**; the lab account, named **example-security-lab**, was **222222222222**.

The reason for separating them was to keep experiments away from the account that manages the organization. This separation reduces accidental mixing of work, but it is not a substitute for careful permissions. A powerful role in a lab account can still change or delete resources in that account.

Protecting the account owner

The root user is the original account-owner identity. It has very broad authority and should not be the routine identity for building an application. The setup established root MFA and avoided root access keys. MFA means multi-factor authentication: signing in requires an additional factor beyond the password. I then used a named Identity Center user for day-to-day lab work.

What the separation helped me understand

I kept administration of the organization separate from routine work inside the member account. Before each important change, I checked the account and role shown in the console header. The same browser can open multiple AWS sessions, so the visible account context mattered.

I retained the organization and the accounts after the exercise. They provide a foundation for future labs; they were not application resources that needed to be destroyed with the test server.

LAB WALKTHROUGH / 04

Signing in with IAM Identity Center

IAM stands for Identity and Access Management. It answers two basic questions: who is making a request, and what are they allowed to do? IAM Identity Center gave me a central sign-in portal rather than a separate permanent user password for every member account.

I enabled the organization instance of Identity Center in **us-east-1**, created **lab.user**, and completed the password and MFA setup. The AWS access portal then let me choose an account and an assigned permission set. The lab identity was different from the root identity used for initial administration.

What a permission set means

A permission set is a named package of permissions for a human user. I think of it as a job description: a viewer can inspect, a builder can create selected resources, and a temporary setup role can perform a narrow administrative task. Assigning a permission set connects a person, an account, and a level of access. [2]

Creating a permission set by itself does not grant access. It also needs an assignment to the user or group in the target account. A permission set marked **Provisioned** has been applied to an account. A set marked **Not provisioned** may still exist as a definition without an active account deployment.

Temporary sessions

My permission sets used a one-hour session duration. After signing in, I worked through an assumed role session. The session had temporary credentials. This is different from writing a long-lived AWS secret into a file. The role name in the console header helped me check which permissions were active before making a change.

When a page denied an action, I checked both the account number and the role name. Being signed in successfully does not mean a requester is authorized to do everything. In this lab, switching between LabBuilder, LabRoleSetup, and LabAuditSetup was a deliberate part of controlling access.

Authentication means proving who a requester is. **Authorization** means deciding what that identity may do. A valid login can still receive AccessDenied because it lacks permission for a specific action.

LAB WALKTHROUGH / 05

The four permission sets I used

PERMISSION SET	PURPOSE	HOW I USED IT
ReadOnlyAccess	Inspection without routine changes	Established for viewing the management account. It was not the application identity.
LabBuilder	Build the EC2 and S3 lab	Used for selected EC2 operations, lab-prefixed S3 resources, and passing LabAppRole to EC2.
LabRoleSetup	Create or remove the application role and profile	Assigned temporarily, removed after setup, restored for cleanup, then removed again.
LabAuditSetup	Create and clean up the lab audit trail and bucket	Assigned for CloudTrail data logging and audit-bucket tasks, then removed after cleanup.

LabBuilder included EC2 permissions limited to the selected region and S3 permissions limited to buckets with the lab naming prefix for many operations. It also allowed account-wide bucket listing. These permissions made the lab workable without granting general AdministratorAccess.

This was not perfect production least privilege. Some builder operations were broad within the lab region. The most tightly scoped permission in the exercise was the application role: it could read only hello.txt. Human setup access and application runtime access served different purposes.

Managed and inline policies

A policy is a set of permission rules. An **AWS managed policy** is maintained by AWS and can be attached to identities or included in permission sets. An **inline policy** is written directly for the particular permission set or role. I used custom inline rules for the lab and added **AWSCloudTrail_ReadOnlyAccess** for CloudTrail inspection.

A read-only CloudTrail policy does not automatically provide every S3 permission the console might need. That distinction later explained why one role could work with a particular audit bucket but could not list all buckets in the account.

LAB WALKTHROUGH / 06

Reading an IAM policy without getting lost

An IAM policy is usually written as JSON. JSON is a text format with named fields and values. It is not a program that runs line by line. AWS reads the rules when a request arrives and decides whether the request is allowed.

FIELD OR IDEA	MEANING IN BASIC TERMS
Version	The policy-language version, commonly 2012-10-17. It is not the creation date of my policy.
Statement	One permission rule, or a list of rules.
Effect: Allow	This rule grants the listed actions when its other requirements match.
Action	The operation, such as s3:GetObject or ec2:RunInstances.
Resource	The specific AWS item to which the rule applies.
Condition	An extra requirement, such as a particular region or service.
ARN	Amazon Resource Name: a formal address for an AWS resource.
Implicit deny	If no applicable permission grants the action, the request is denied.
Explicit deny	A Deny rule overrides an Allow that would otherwise apply.

Why exact names and resource types matter

The bucket ARN ends at the bucket name. An object ARN adds a slash and the object key. **s3:ListBucket** applies to the bucket; **s3:GetObject** applies to an object. Granting one does not grant the other. A wildcard such as * matches a broader set of values, so using it carelessly can grant more access than intended.

What iam:PassRole did

The builder needed permission to attach LabAppRole to the EC2 instance. That is the purpose of **iam:PassRole**. It does not mean the builder automatically becomes the role. It means the builder may give an approved service a specified role. I limited this to LabAppRole and, in the builder policy, to passing it to EC2.

This matters because attaching an overly powerful role to a server could give software on that server more AWS access than the human should be able to delegate. Restricting which role can be passed helps close that gap.

LAB WALKTHROUGH / 07

LabAppRole: the application’s AWS identity

I created an IAM role named **LabAppRole**. The role had two different pieces: a trust policy and a permissions policy. The trust policy answered who could use the role. The permissions policy answered what the role could do once used.

The trust policy

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Service": "ec2.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}
```

This trust relationship allowed the EC2 service to assume the role. STS means Security Token Service. Assuming a role means obtaining temporary credentials for that role, not copying a permanent password.

The permissions policy

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource":
      "arn:aws:s3:::example-security-lab-222222222222-data/hello.txt"
  }]
}
```

The inline policy, named **LabAppRolePolicy**, allowed one operation on one object. It did not allow listing the bucket, uploading files, deleting files, or reading every object in the account. This is an example of least privilege: give the application only what its task requires.

During role setup, the console also needed **access-analyzer:ValidatePolicy** to validate the policy text. I added that specific action to the temporary setup permissions. Policy validation checks a policy; it does not grant the application more S3 access.

An **instance profile**, also named LabAppRole, connected the IAM role to the EC2 instance. Think of the profile as the attachment that lets EC2 use the role. The SDK could then obtain temporary credentials through the instance role mechanism. No aws configure step with permanent keys was needed. [3]

LAB WALKTHROUGH / 08

Creating the private S3 data bucket

Amazon S3 is object storage. It stores files as objects inside buckets. Each object has a **key**, which is its name or path inside the bucket. My bucket was **example-security-lab-222222222222-data**, and the object key was **hello.txt**.

I chose a general purpose bucket in US East (N. Virginia), also called **us-east-1**. The account number in the name helped make the bucket name distinctive. A bucket name is an identifier, not a secret or an access credential.

SETTING	WHAT WE SELECTED	WHY
Block Public Access	All options enabled	Helped prevent accidental public access through supported bucket policies and ACL settings.
Object Ownership	Bucket owner enforced; ACLs disabled	Kept ownership and access control centered on policies rather than separate object ACLs.
Default encryption	SSE-S3	S3 encrypted stored object data using S3-managed keys.
Versioning	Disabled	Kept the small lab simple. I did not retain multiple saved versions of hello.txt.
Object Lock	Not enabled	I did not require a retention lock for this disposable test file.

Encryption at rest protects stored data. It is different from permissions. An encrypted object can still be readable by someone who has access. **Encryption in transit** protects a connection, such as HTTPS to S3. These are separate controls.

LAB WALKTHROUGH / 09

Uploading hello.txt and testing privacy

I uploaded a small text file named hello.txt. Its contents were:

```
Hello from Harry's private S3 lab.
```

The later CloudTrail event reported an object size of **34 bytes**, matching this text without a trailing newline. A byte is a unit of data size. Keeping the object tiny made the application behavior and audit evidence easy to understand.

Two different ways of trying to read it

Using the authorized AWS identity, I could retrieve the file. Opening its ordinary S3 object URL without authentication returned **AccessDenied**. A normal object URL tells the browser where the object is; it does not supply permission to read a private object.

This was a **negative test**: I intentionally tried something that should fail. Security testing needs both positive tests, which show intended access works, and negative tests, which show unwanted access is blocked.

What the test did and did not prove

The test showed that this unauthenticated request could not read this object at that time. It did not prove every object or policy in the whole AWS account was secure. Its scope was deliberately narrow.

A private bucket can still be read by an authorized application. That is the design I wanted: S3 did not need to become public just because an application needed to read from it. The role supplied the authorization.

Later, the browser received the same text from the API. That was a separate path: the application fetched the private object and returned a response. It did not turn the S3 bucket into a public bucket.

LAB WALKTHROUGH / 10

Launching the EC2 Linux server

Amazon EC2 provides virtual computers called instances. I launched one named **example-lab-api**. The Name tag was a convenient label; the unique instance ID was **i-0123456789abcdef0**. A tag is a key-value label that helps identify and organize a resource.

CHOICE	LAB VALUE	MEANING
Operating system	Amazon Linux 2023	The Linux software environment on the cloud computer.
Instance type	t2.micro	The size and family of the virtual machine.
AMI	ami-0123456789abcdef0	The machine image used as the starting template.
Region / zone	us-east-1 / us-east-1d	The AWS geographic region and one availability zone within it.
Root storage	8 GiB gp3, encrypted	The disk that held Linux, Python, and the application.
Disk cleanup	Delete on termination enabled	The root volume was set to be removed with the instance.
IAM instance profile	LabAppRole	Attached the application’s AWS permissions to the server.
Metadata access	IMDSv2 required; hop limit 1	Required token-based instance metadata requests.

An **EBS volume** is a virtual disk attached to an EC2 instance. gp3 identifies a general purpose SSD volume type. Encryption was enabled for the launch. The root volume is different from S3: it acts like the server’s disk, while S3 is an object-storage service reached through an API.

IMDS is the Instance Metadata Service. It supplies information about the instance and access to its role credentials. IMDSv2 uses a token-based request process. Requiring it adds protection against some unwanted metadata-access paths, but it does not make the application immune to compromise.

My downloaded successful RunInstances event confirmed the instance ID, the attached role profile, the encrypted 8 GiB disk request, and the metadata settings. An early response state of **pending** was normal while the instance was starting.

LAB WALKTHROUGH / 11

Networking: addresses, ports, and the firewall

The instance used the default VPC and a subnet in the lab account. A **VPC** is a private network inside AWS. A **subnet** is a smaller address range within that network. The instance had private address **10.0.1.10** and public address **203.0.113.10**. The public address was used from my computer during the lab.

A **port** identifies a listening service on a computer. I used TCP port **22** for SSH administration and TCP port **8080** for the application. TCP is the transport protocol used by those connections.

INBOUND RULE	SOURCE	PURPOSE
SSH, TCP 22	198.51.100.10/32	Let my then-current public IP reach the SSH server.
Custom TCP, 8080	198.51.100.10/32	Let the same public IP reach the test API.

A **security group** is an AWS network firewall attached to resources such as an instance’s network interface. Inbound rules control which connections may enter. The /32 suffix means exactly one IPv4 address. It does not identify a person or a specific laptop: other devices sharing that public address could match the same rule.

I did not open port 8080 to every internet address. I also did not add ports 80 or 443 just because they are common web ports. The rule had to match the port actually used by Uvicorn. Network permission and application listening settings both had to be correct.

LAB WALKTHROUGH / 12

SSH access and fixing the private-key error

SSH means Secure Shell. It provides an encrypted command-line connection to the Linux server. I used a key pair named **example-lab-key**. The private key stayed on my Windows computer as a .pem file. The matching public-key information allowed the instance to recognize a valid login.

```
ssh -i "$HOME\Downloads\example-lab-key.pem" ec2-user@203.0.113.10
```

The -i option selects the private-key file. ec2-user is the Linux login name for this Amazon Linux instance. The address after @ is the destination server. The first SSH connection also presented a host-key prompt, which concerns the server’s identity rather than my user key.

Why the first attempt failed

SSH reported that the private-key permissions were too open. It ignored the key because other local users or groups could access it. A private key should be readable only by appropriate trusted identities. The problem was a Windows file-permission issue, not a broken EC2 role.

```
$labKey = "$HOME\Downloads\example-lab-key.pem"
$labUser = whoami
icacls.exe "$labKey" /reset
icacls.exe "$labKey" /grant:r "${labUser}:(R)"
icacls.exe "$labKey" /inheritance:r
```

These commands targeted the key file only. whoami identified the current Windows user. The grant gave that user read access. Removing inherited permissions stopped broader parent-folder permissions from applying to the key. After this change, SSH succeeded.

LAB WALKTHROUGH / 13

Proving which identity the server used

After connecting to EC2, I checked the identity behind AWS CLI requests. The AWS CLI is a command-line client for AWS services. It allows a user to perform operations without using the web console.

```
aws sts get-caller-identity
```

The response showed an assumed-role ARN containing **LabAppRole** and the instance ID. This was important: the server was using the application role, not my human LabBuilder session. The account number also confirmed the request was running in the intended lab account.

The allowed read test

```
aws s3 cp s3://example-security-lab-222222222222-data/hello.txt ./hello.txt
cat hello.txt
```

The cp command copied the named S3 object into a local file on EC2. The leading ./ means the current directory. cat printed the file contents. I saw the greeting, so the role could perform its intended task.

The denied listing test

```
aws s3 ls s3://example-security-lab-222222222222-data/
```

This returned AccessDenied for **s3:ListBucket**. That was expected. Knowing an exact object name and reading it is different from asking S3 to show the bucket’s contents. The role could GetObject but had no permission to ListBucket. I did not broaden the role merely to make that negative test succeed.

LAB WALKTHROUGH / 14

Preparing Python and a virtual environment

Python is the programming language used for the API. The server initially had Python 3.9, but its pip module was missing. **pip** is a package installer: it downloads and installs Python libraries. A library is reusable code that supplies functionality the application needs.

I installed Python 3.11 and its package tooling, created a project directory, and made a virtual environment. A **virtual environment** is a separate set of Python packages for one project. It helps avoid mixing the application's dependencies with the operating system's Python packages.

```
mkdir -p ~/example-lab-api
cd ~/example-lab-api
python3.11 -m venv .venv
source .venv/bin/activate
python --version
python -m pip --version
```

The tilde (~) means the current Linux user's home directory. mkdir -p creates the directory if needed. cd moves into it. The -m option runs a Python module. source activates the environment in the current shell, which is why the prompt began with (.venv).

I installed **FastAPI**, **Uvicorn**, and **Boto3**. FastAPI defined the web interface, Uvicorn served it, and Boto3 talked to AWS. Their dependencies were installed too; a long installation list was normal.

```
python -m pip install fastapi uvicorn boto3
python -c "import fastapi, uvicorn, boto3; print('All three packages imported successfully')"
```

The import test succeeded. Installing a library and successfully importing it are useful checks, but they do not yet prove that the application can reach S3. I tested that separately through /hello.

LAB WALKTHROUGH / 15

What the API did

An API, or Application Programming Interface, is a defined way for software to request a result. My API used HTTP, the same request-and-response protocol used by web browsers. Each route, also called an endpoint, identified a particular behavior.

ENDPOINT	EXPECTED RESPONSE	WHAT IT TESTED
/health	{"status":"ok"}	The web application was running and could answer a simple request.
/hello	{"message":"Hello from Harry's private S3 lab."}	The application could retrieve the configured object from S3 and return its text.

The responses used JSON. In {"status":"ok"}, status is a field name and ok is its value. This predictable structure is easy for another program to read. A browser can display JSON directly, even though it is not a styled webpage.

The application’s main steps

• Create a FastAPI application object.

• Create an S3 client using Boto3 and the lab region.

• For /health, return a small fixed response.

• For /hello, request the fixed bucket and key with get_object.

• Read the returned byte stream, decode it as UTF-8 text, and close the stream.

• Return the text in a message field. If an AWS request fails, log the error and return a generic failure response.

UTF-8 is a way of representing text as bytes. S3 returns object bytes; decoding changes those bytes into the Python text string placed in JSON. Closing the stream releases the underlying network resource.

The code used a fixed object name rather than accepting an arbitrary bucket or key from visitors. This kept the exercise focused. It did not turn the API into a general-purpose S3 reader.

The application did not store a permanent AWS access key or secret. Boto3 used the instance role credentials. Error details were intended for server logs, while the visitor received a generic 502 message if retrieval failed. A 502 is an HTTP error status indicating that this API could not successfully complete its upstream retrieval.

LAB WALKTHROUGH / 16

A readable version of the application

Public source excerpt: this is my saved application code with only the bucket name replaced by an example value. The repository includes the same sanitized code as app.py.

```
import logging

import boto3
from botocore.exceptions import BotoCoreError, ClientError
from fastapi import FastAPI, HTTPException

app = FastAPI(title="Harry's AWS Security Lab")
s3 = boto3.client("s3", region_name="us-east-1")

BUCKET = "example-security-lab-222222222222-data"
KEY = "hello.txt"


@app.get("/health")
def health():
    return {"status": "ok"}


@app.get("/hello")
def hello():
    try:
        response = s3.get_object(Bucket=BUCKET, Key=KEY)
        body = response["Body"]
        try:
            message = body.read().decode("utf-8")
        finally:
            body.close()
        return {"message": message}
    except (ClientError, BotoCoreError):
        logging.exception("Unable to read the lab object")
        raise HTTPException(
            status_code=502,
            detail="Unable to retrieve the lab file",
        )
```

The @app.get lines connect a URL path to a Python function. GET is the HTTP method used to request a resource. return sends the function result back to FastAPI, which converts the dictionary into a JSON response.

try and except handle anticipated AWS failures. finally closes the response body even if reading it fails. ClientError represents an AWS service error, such as denied access. BotoCoreError represents errors from the AWS client machinery. These checks make failures understandable; they do not replace a complete production error-handling design.

LAB WALKTHROUGH / 17

Testing locally, then from my browser

First, listen only on the server itself

```
python -m uvicorn app:app --host 127.0.0.1 --port 8080
```

The first app in app:app meant the app.py module. The second app meant the FastAPI object inside that module. The address 127.0.0.1 is loopback: it refers to the same computer. While bound there, the application could be tested from EC2 but could not be reached directly through the instance's public address.

```
curl -i http://127.0.0.1:8080/health
curl -i http://127.0.0.1:8080/hello
```

curl is a command-line HTTP client. The -i option includes response headers. Both tests returned **HTTP 200 OK**, the normal success status. The first proved the process answered; the second also exercised the S3 retrieval path.

Then, listen on the network interfaces

```
python -m uvicorn app:app --host 0.0.0.0 --port 8080
```

Binding to 0.0.0.0 means listening on all IPv4 interfaces on the instance. It is not an address to type into a remote browser. From my computer, I used the instance's public address, 203.0.113.10. The security group still controlled which incoming connections could reach the listening process.

This order helped isolate problems. If the local test failed, the application or S3 access needed attention first. If local testing passed but remote testing failed, the listening address and network rules became the next things to inspect.

LAB WALKTHROUGH / 18

Troubleshooting and the remote security test

The SSH connection reset

The terminal showed “Connection reset,” so I reconnected. A dropped SSH session is a connection event; it does not prove the instance stopped or that every program launched from the session ended. In this case, the old Uvicorn process remained running.

“python: command not found” after reconnecting

A new shell did not automatically reactivate the project environment. I moved into ~/example-lab-api and ran source .venv/bin/activate again. The (.venv) prompt helped confirm that python referred to the project environment.

“Address already in use”

```
ss -ltnp 'sport = :8080'
```

ss displayed listening network sockets. The output identified a Python process with PID 27739 on 127.0.0.1:8080. PID means process identifier. I stopped that specific old process with kill 27739, then restarted Uvicorn on 0.0.0.0. I did not need a second instance or a wider security-group rule.

Remote success and denied-source testing

My browser successfully loaded /health and /hello through the public address. I then observed that a test from a different connection timed out, while the allowed connection still worked. That result was consistent with the source-IP restriction. A timeout alone would not prove the cause, so the successful allowed-source control test mattered.

A favicon.ico request returned 404 because I had not created a browser-icon route. This did not mean /hello failed. I also did not add HTTPS, application login, a process supervisor, or automatic service restart during this lab.

LAB WALKTHROUGH / 19

CloudTrail: recording who did what

CloudTrail is an AWS activity record. It can answer questions such as: which identity made a request, which operation did it request, when did it happen, and did AWS return an error? This is called an audit trail because it helps someone review activity after the fact.

Management events versus data events

TYPE	EXAMPLE FROM THIS LAB	WHY IT MATTERS
Management event	RunInstances or AuthorizeSecurityGroupIngress	Records changes to the infrastructure or configuration.
Data event	GetObject for hello.txt	Records access to data inside a resource, such as reading an S3 object.

Event history showed recent management events for the selected region. It covers the past 90 days of recorded management events. S3 object data events are not shown in Event history, so finding the instance-launch event did not mean I was already recording hello.txt reads. [4]

I added AWSCloudTrail_ReadOnlyAccess to LabBuilder so I could inspect CloudTrail. Read-only access made it possible to review evidence without granting general authority to change logging configuration.

Why I downloaded the records

A screenshot is useful for showing what the console looked like. The JSON record is more useful for checking exact fields, times, identities, and resource IDs. I downloaded the management-event evidence so it would remain available after deleting the temporary lab infrastructure.

CloudTrail records AWS API activity. It is not a packet capture of all network traffic, and it is not a replacement for the application’s own web-request logs. Each evidence source answers different questions.

LAB WALKTHROUGH / 20

Understanding the management-event evidence

Three RunInstances records at **00:18:03**, **00:18:57**, and **00:22:26 EDT** contained Client.DryRunOperation and the message “Request would have succeeded, but DryRun flag is set.” These were permission-check attempts, not three extra running instances.

A dry run asks whether the request would be allowed without carrying out the actual launch. In these records, the word errorCode was easy to misread. The message explained that this particular response represented a successful dry-run authorization check. It was not evidence of an unauthorized operation.

The actual instance launch

The successful launch occurred at **2026-10-15T04:40:38Z**, which was **00:40:38 EDT**. The response included instance **i-0123456789abcdef0**. The initiating identity was the lab.user session of the Identity Center LabBuilder role, not the runtime LabAppRole.

FIELD	WHAT THE RECORD SHOWED
eventName	RunInstances: the EC2 launch operation.
userIdentity	An assumed human LabBuilder role session named lab.user.
sourceIPAddress	198.51.100.10, the source address used for the console action.
requestParameters	t2.micro, the key pair, encrypted storage settings, role profile, and metadata options.
responseElements	An actual instance ID and an initial pending state.
eventID	EXAMPLE-LAUNCH-EVENT-ID.

The security-group change

At **01:33:10 EDT**, an AuthorizeSecurityGroupIngress event recorded the TCP 8080 rule for **198.51.100.10/32** in security group **sg-0123456789abcdef0**. The response had _return: true and rule ID **sgr-0123456789abcdef0**. This connected the visible firewall setting to the identity that created it.

The log therefore supported more than “the API worked.” It helped show how the environment was created and who opened the specific inbound path used for the test.

LAB WALKTHROUGH / 21

Creating a trail for S3 object reads

To record the actual file read, I created a separate temporary permission set, **LabAuditSetup**. It allowed management of the named lab trail and its audit bucket. I assigned it to lab.user in the lab account, then signed in through that permission set.

I created the trail **example-lab-s3-audit** and the destination bucket **example-security-lab-222222222222-audit**. The data bucket held the application's file; the audit bucket held the evidence about access to that file. Keeping those purposes separate made the evidence easier to understand.

TRAIL SETTING	LAB CHOICE
Home region	US East (N. Virginia), us-east-1.
Multi-region trail	Yes, as shown on the created trail.
Organization trail	No. This was a trail for the lab account.
Data events	Enabled with a narrowly targeted S3 selector.
Management events for this trail	Left out of the final data-focused configuration; management history was reviewed separately.
Log-file validation	Enabled to support checking log integrity.
CloudWatch Logs / SNS / Insights	Not configured for this exercise.
SSE-KMS option	Not selected. I did not add customer-managed KMS-key administration.

Log-file validation supports detecting changes to delivered logs when validation is performed. Turning it on is not the same as having completed an integrity-validation exercise. I downloaded and inspected the event record; I did not claim to have run a full cryptographic validation process.

LAB WALKTHROUGH / 22

Choosing exactly which data events to log

A trail can produce a lot of records. I used advanced event selectors to narrow this lab’s data logging to reads of the one object I cared about. A selector is a filter that decides which events should be recorded; it does not grant or deny access to S3.

FIELD	MATCH	MEANING
eventCategory	Equals Data	Capture data activity rather than infrastructure configuration activity.
resources.type	Equals AWS::S3::Object	Focus on S3 objects.
resources.ARN	Equals the full hello.txt object ARN	Limit the matching resource to the exact lab file.
readOnly	Equals true	Capture matching read activity rather than writes.

The selector name was **ReadHelloObject**. The name itself is only a label. Earlier, the object ARN had been entered in the selector-name box, but that alone would not filter events. The ARN needed to be in the resources.ARN condition.

I used Equals, so the ARN matched one object instead of a broader prefix. The selector did not explicitly say eventName equals GetObject. It matched read-only data events for that object; the event I later observed was GetObject. This is a subtle but important distinction.

I left optional event aggregation unconfigured. Data-event logging can incur charges, so a focused selector and a short logging period helped keep the lab bounded. [5]

LAB WALKTHROUGH / 23

The strongest evidence: the actual GetObject

After the trail was logging, I requested /hello again. The browser returned the expected message. I then checked the audit bucket. At first, the CloudTrail prefix was empty. Log delivery is not immediate, so an empty listing right after a test was not enough to conclude that logging was broken.

A log file was eventually delivered and downloaded. Its Records array contained a GetObject event for the exact file. My downloaded JSON was direct evidence of the S3 request, rather than just a screenshot of the final greeting.

RECORDED FIELD	VALUE AND MEANING
eventTime	2026-10-15T16:55:30Z, or 12:55:30 PM EDT.
eventName / source	GetObject / s3.amazonaws.com.
Role session	LabAppRole/i-0123456789abcdef0: the application role associated with the instance.
Bucket and key	example-security-lab-222222222222-data / hello.txt.
sourceIPAddress	203.0.113.10, the instance’s public address.
HTTP status	200, recorded in additionalEventData.
Object and bytes out	34 bytes for the object size and bytes transferred out.
Transport	TLSv1.3; authentication used a signed AWS request.
Event category	Data; readOnly true; managementEvent false.
eventID	EXAMPLE-READ-EVENT-ID.

The event had responseElements: null. That did not mean the read failed. I used the event’s httpStatusCode of 200 and transfer fields, together with the successful application response, to interpret the result.

The user agent identified Boto3 and Python. The record also indicated credentials issued to the EC2 instance and ec2RoleDelivery 2.0. Together, these fields connected the application request to the instance role. I did not need to expose or save a secret access key to demonstrate this behavior.

The source address in this event was the EC2 server, not my laptop. That makes sense: the browser called the API, and the API made a separate call to S3. [6]

LAB WALKTHROUGH / 24

Saving evidence and reconstructing the timeline

Before removing the infrastructure, I saved the application code and audit evidence. Cleanup should remove the temporary resources without destroying the record of what I learned and tested.

The code download

```
scp -i "$HOME\Downloads\example-lab-key.pem" `
  ec2-user@203.0.113.10:/home/ec2-user/example-lab-api/app.py `
  "$HOME\Downloads\example-lab-api.py"
```

This PowerShell example uses backticks to continue the command across lines. scp copies a file over SSH. The source was app.py on EC2, and the destination was a file in Downloads on my Windows computer. I saved the download.

Important times from the record

EDT ON OCTOBER 15	EVENT	EVIDENCE
00:18-00:22	Three EC2 dry-run checks	Saved CloudTrail JSON.
00:40:38	Actual EC2 launch	RunInstances response with the instance ID.
01:33:10	Port 8080 rule added	AuthorizeSecurityGroupIngress JSON.
12:52:42	Audit bucket creation time	Trail setup/bucket listing context.
12:55:30	Application read of hello.txt	Delivered GetObject data event.
Afternoon	Logging stopped; infrastructure removed	My recorded cleanup steps.

I did not record the exact minute of every action, so I have left those times approximate. Console screenshots may show local time while the JSON uses UTC. I had to convert time zones before comparing records.

The evidence set included screenshots of configuration and tests, downloaded management-event records, the delivered S3 data-event JSON, and the saved Python application. I kept this repository focused on the AWS exercise.

LAB WALKTHROUGH / 25

Cleanup: removing resources in a sensible order

Cleanup was part of the security exercise. A server that is no longer needed can still cost money and remain reachable. Unnecessary roles and temporary setup assignments also leave access behind. I saved the useful evidence first, then removed the temporary infrastructure.

RESOURCE OR STEP	WHAT I DID	FINAL EVIDENCE STATUS
CloudTrail logging	Stopped the lab trail’s logging.	Completed during my cleanup.
EC2 instance	Terminated example-lab-api.	Terminated during my cleanup.
EBS storage	Checked that no lab volume remained.	I checked that the volumes view was empty.
Lab security group and key pair	Deleted the lab-specific entries.	Completed during my cleanup.
Data bucket	Deleted hello.txt, then the bucket.	Later bucket listing showed only the audit bucket.
Audit bucket	Removed the saved-log copies in S3 and deleted the bucket.	Deleted during my cleanup.
CloudTrail trail	Checked and deleted example-lab-s3-audit.	Completed during my cleanup.

Stopping an instance and **terminating** it are different. Stopping keeps the instance available to restart and can leave chargeable storage. Terminating removes the instance. Because the root volume was configured for deletion on termination, I expected it to disappear too, and I checked the volumes view.

The public address was automatically assigned rather than a separately allocated Elastic IP in this lab. I did not create a custom machine image or my own snapshot to keep. The image’s source snapshot in the launch record was not a new backup created by this lab.

I kept the default network foundation, the organization, and the accounts. Those were not disposable application resources. I also kept the downloaded local evidence. Deleting a trail does not automatically delete the log objects it previously wrote, which is why the audit bucket required separate cleanup.

LAB WALKTHROUGH / 26

Cleanup permission errors and the IAM finish

Why the Empty bucket button was blocked

The console reported that **s3:ListBucketVersions** was missing. The console’s Empty workflow needs to inspect versions. That permission is separate from ordinary bucket listing. The data bucket had versioning disabled, so deleting the known object and then deleting the empty bucket avoided broadening the builder policy simply to use that button.

Why a bucket list showed zero with an error

Under LabAuditSetup, the general-purpose bucket page reported missing **s3:ListAllMyBuckets**. The page’s zero count did not prove that all buckets were deleted. I switched to LabBuilder, which had listing access. Its successful listing showed only the audit bucket, confirming that the data bucket was gone.

A separate Vector buckets page reported missing s3vectors:ListVectorBuckets. Vector buckets were not used in this lab, so that message did not require adding more permissions. I returned to the relevant general-purpose bucket view.

Removing the application role

I temporarily reassigned LabRoleSetup, signed into the lab account through it, and deleted **LabAppRole** in IAM. I completed the deletion. For this console workflow, the same-name instance profile and inline role policy are removed with the role. [7]

I then removed the **LabRoleSetup** and **LabAuditSetup** assignments for lab.user in the lab account. I retained **LabBuilder** for final checks. I removed the assignments rather than deleting every permission-set definition. That does not prove that all previously issued sessions ended immediately.

What I learned and what remains

The most useful result was not simply seeing a greeting in a browser. It was understanding why the application could retrieve the file, why a different S3 action was denied, and how I could connect that behavior to an AWS identity in the audit log.

The results I demonstrated

• I used a named human identity with assigned roles for routine lab work.

• My EC2 application retrieved a private S3 object through its instance role without permanent AWS credentials in the application.

• The same application role could read hello.txt but could not list the bucket.

• The API responded to local tests and to the allowed remote source. My alternate-connection test timed out while the allowed source still worked.

• I inspected management records for the launch and firewall change, then captured a GetObject data event for the application read.

• I saved the evidence and completed cleanup of the temporary lab infrastructure and setup assignments.

What I would add before a production deployment

I did not configure browser-to-API HTTPS, application authentication, a service supervisor, automated deployment, high availability, or a broad security assessment. Binding to all interfaces and allowing one source address was sufficient for this controlled exercise, but it was not a complete production access model.

The CloudTrail selector covered reads of a specific object. It did not establish account-wide monitoring. Log validation was enabled, but I did not perform a separate full cryptographic validation exercise.

Access I retained

I retained LabBuilder for final checks and kept the account foundations and existing read-only access. I removed the temporary LabRoleSetup and LabAuditSetup assignments. I am not claiming that the account was closed, that all identity definitions were deleted, or that assignment removal instantly revoked every existing session.

My next improvement would be to rebuild the environment from reviewed infrastructure code, add HTTPS and an application access policy, and repeat the same positive and negative tests. Those are future steps, not features completed in this lab.

REFERENCE / 28

Glossary: identities, storage, and networking

TERM	SIMPLE DEFINITION
AWS account	A container for AWS resources, permissions, and billing relationships.
AWS Organizations	A service for grouping and managing multiple AWS accounts.
Root user	The original account-owner identity with broad authority.
MFA	An additional sign-in factor beyond a password.
IAM	The system that manages AWS identities and their permissions.
Identity Center	A central sign-in and access-assignment service for people.
Permission set	A named collection of permissions assigned to a human user or group in an account.
IAM role	An identity that can be assumed to obtain temporary permissions.
Trust policy	The role policy specifying who or what may assume the role.
Permissions policy	Rules specifying allowed or denied AWS operations on resources.
Instance profile	The mechanism used to attach an IAM role to EC2.
Least privilege	Granting only the access needed for a task.
ARN	A structured AWS resource identifier.
S3 bucket / object / key	A storage container, an item stored in it, and that item’s name.
ACL	An access-control list. Object ACLs were disabled in the lab bucket.
SSE-S3	S3 server-side encryption using S3-managed keys.
Versioning	Keeping multiple versions of objects instead of only the current value.
VPC / subnet	An AWS private network and a smaller address range inside it.
Security group	A network firewall associated with AWS resources.
CIDR /32	Notation selecting one IPv4 address.
Public / private IP	An internet-routable address versus an address used within private networking.
Port	A number identifying a network service on a computer.

REFERENCE / 29

Glossary: software, logs, and test results

TERM	SIMPLE DEFINITION
EC2 / instance	AWS virtual-computer service and one virtual computer within it.
AMI	A template used to launch an instance with an operating system.
EBS / gp3	EC2 disk storage and a general purpose SSD volume type.
IMDSv2	The token-based version of EC2 instance metadata access.
SSH / key pair	An encrypted remote terminal connection and the cryptographic keys used for login.
Shell / PowerShell	A command interpreter; PowerShell was used on Windows.
Python / pip / venv	The programming language, its package installer, and an isolated project environment.
FastAPI / Uvicorn	The web framework and the server that ran it.
Boto3 / SDK	The AWS Python library; an SDK is a software development kit.
API / endpoint	A defined software interface and a particular route within it.
HTTP / HTTPS / TLS	Web messaging, web messaging protected by TLS, and the transport-encryption protocol.
JSON	A structured text format built from fields, values, arrays, and objects.
GET / 200 / 404 / 502	A read request; success; route or resource not found; an upstream failure response used by the API.
Loopback / 0.0.0.0	The local computer only; a listening address covering all IPv4 interfaces.
PID / listening socket	A process identifier and a network endpoint waiting for connections.
CloudTrail / trail	AWS activity records and a configured delivery stream for selected events.
Management / data event	An infrastructure-level operation versus an operation on data within a resource.
Selector	A rule that chooses which events are logged.
Dry run	A check that does not carry out the requested resource operation.
Positive / negative test	An action expected to succeed versus an action expected to be denied.
UTC / EDT / Z	Universal time; Eastern daylight time; the UTC suffix in timestamps.
Terminate / delete	Remove the instance or resource, rather than merely stop its operation.

REFERENCE / 30

Command reference and where each command ran

These are historical lab commands, not instructions to run against the deleted server. The public address may be reassigned after termination. For a future lab, use that lab's current addresses and resource names.

Windows PowerShell: connect and copy

```
ssh -i "$HOME\Downloads\example-lab-key.pem" ec2-user@203.0.113.10

scp -i "$HOME\Downloads\example-lab-key.pem" `
  ec2-user@203.0.113.10:/home/ec2-user/example-lab-api/app.py `
  "$HOME\Downloads\example-lab-api.py"
```

SSH opened a remote shell. SCP downloaded the code before the server was removed. The private key was a login credential and was not an application source file or a report attachment.

EC2 shell: identity and object access

```
aws sts get-caller-identity
aws s3 cp s3://example-security-lab-222222222222-data/hello.txt ./hello.txt
cat hello.txt
aws s3 ls s3://example-security-lab-222222222222-data/
```

The identity check showed LabAppRole. The copy and print succeeded. The final listing command was deliberately denied because ListBucket was not allowed.

EC2 shell: start, test, and diagnose the application

```
cd ~/example-lab-api
source .venv/bin/activate
python -m uvicorn app:app --host 0.0.0.0 --port 8080

# Run tests from a separate SSH session while the server is running:
curl -i http://127.0.0.1:8080/health
curl -i http://127.0.0.1:8080/hello
ss -ltnp 'sport = :8080'
```

Uvicorn occupied the terminal while serving requests. A separate SSH session let me run curl and diagnostics without interrupting it. kill 27739 was used only after identifying that specific old process; a PID should never be assumed to identify the same process in a future session.

REFERENCE / 31

Evidence register and references

I kept the original evidence privately. The repository contains three sanitized evidence files: run-instances.sanitized.json, get-object.sanitized.json, and management-checks.sanitized.json in the evidence folder. Each file labels itself as an edited excerpt and lists the identifiers that were replaced or omitted.

I reviewed my saved application source for publication. The repository contains that source as app.py with the identifying bucket name replaced by the same example value used throughout this report. Its request handling and error handling are unchanged.

What the excerpts show

EXCERPT	WHAT IT SUPPORTS
RunInstances	A successful EC2 launch with the application profile, encrypted storage request, and IMDSv2 settings.
GetObject	A successful read of the intended object by the instance role, with HTTP 200 and 34 bytes returned.
Management checks	Selected fields from three dry runs and the TCP 8080 ingress change, transcribed from my saved session records.

My account and resource placeholders preserve the relationships that matter to the explanation. I omitted credential identifiers, internal principal IDs, request IDs, account-directory references, and unneeded network details. I did not include credentials, a private key, or the unredacted screenshots.

Official references

[2] Identity Center permission-set concepts

[3] IAM roles for applications on EC2

[4] S3 CloudTrail events and Event history

[5] Logging CloudTrail data events

[6] Identifying S3 requests using CloudTrail

[7] Deleting IAM roles and instance profiles

These references support the service explanations. My own test outputs and downloaded records support the lab outcomes. This public report documents a guided learning exercise completed in October 2026.